

Block Ciphers

Module 3, Part 2: AES, Integrity, and Stream Ciphers
CSC 3114 Cryptology and Coding Theory

Denish Azamuke, PhD

Office: CoCIS, Block B, Level 3

September 14, 2026

Lecture Outline

- 1 Advanced Encryption Standard (AES)
- 2 AES internal transformations
- 3 Block cipher modes of operation
- 4 Secure communication and message integrity
- 5 Hash functions and Message Authentication Codes
- 6 Stream ciphers

Module 3 Split: Where We Are

Part 1: Completed

- Block cipher model
- Confusion and diffusion
- Feistel networks
- DES structure
- DES limitations

Part 2: Today

- AES as the modern block cipher
- Modes for long messages
- Integrity and authentication
- Hash functions and MACs
- Stream ciphers

Learning Outcomes

By the end of this lecture, you should be able to:

- Describe the high-level structure of AES
- Explain SubBytes, ShiftRows, MixColumns, and AddRoundKey
- Explain why block ciphers need modes of operation
- Distinguish confidentiality from message integrity
- Describe the role of hash functions and MACs
- Explain the basic operation of stream ciphers

From DES to AES

DES was important historically, but its effective key length of 56 bits is too small for modern security.

AES was adopted as a modern replacement.

Main transition

DES uses a **Feistel network**; AES uses a **substitution–permutation network**.

Key takeaway

A modern cipher should combine efficient implementation, strong diffusion, nonlinear substitution, and sufficiently large keys.

AES at a Glance

Property	AES
Block size	128 bits
Key sizes	128, 192, or 256 bits
Structure	Substitution–permutation network
State representation	4×4 byte matrix
Rounds	10, 12, or 14
Common use	Secure communication, storage, VPNs

Observation

The block size is fixed at 128 bits, but the key size can vary.

AES State Representation

AES treats a 128-bit block as 16 bytes arranged in a 4×4 state matrix:

$$\text{State} = \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}.$$

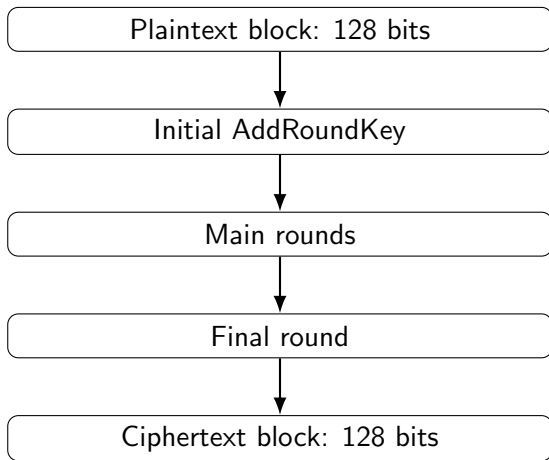
Each entry is one byte:

$$s_{i,j} \in \{0, 1\}^8.$$

Why a matrix?

The matrix view makes it easier to describe row shifts, column mixing, and byte substitution.

High-Level AES Encryption



For AES-128, there are 10 rounds in total.

AES Round Transformations

A normal AES round applies four transformations:

- 1 **SubBytes**: substitute each byte using an S-box.
- 2 **ShiftRows**: cyclically shift rows of the state.
- 3 **MixColumns**: mix bytes within each column.
- 4 **AddRoundKey**: XOR the state with a round key.

$$\text{State} \leftarrow \text{AddRoundKey}(\text{MixColumns}(\text{ShiftRows}(\text{SubBytes}(\text{State}))))).$$

Final round

The final AES round omits MixColumns.

SubBytes: Nonlinear Substitution

SubBytes replaces every byte in the AES state using a fixed S-box.

$$s_{i,j} \mapsto S(s_{i,j}).$$

- It is applied independently to all 16 bytes.
- It introduces nonlinearity.
- It contributes to confusion.

Security role

Without nonlinear substitution, the cipher would be much easier to model and attack algebraically.

ShiftRows: Row-Level Diffusion

ShiftRows cyclically shifts the rows of the AES state.

$$\begin{bmatrix} a_0 & a_1 & a_2 & a_3 \\ b_0 & b_1 & b_2 & b_3 \\ c_0 & c_1 & c_2 & c_3 \\ d_0 & d_1 & d_2 & d_3 \end{bmatrix} \longrightarrow \begin{bmatrix} a_0 & a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 & b_0 \\ c_2 & c_3 & c_0 & c_1 \\ d_3 & d_0 & d_1 & d_2 \end{bmatrix}.$$

Purpose

ShiftRows moves bytes across columns so that later column mixing spreads their influence.

MixColumns: Column-Level Diffusion

MixColumns transforms each column of the state.

Each output byte depends on all four input bytes in the same column:

$$\begin{bmatrix} s'_{0,j} \\ s'_{1,j} \\ s'_{2,j} \\ s'_{3,j} \end{bmatrix} = M \begin{bmatrix} s_{0,j} \\ s_{1,j} \\ s_{2,j} \\ s_{3,j} \end{bmatrix}.$$

- The operation is performed in a finite field.
- It spreads changes within each column.
- Together with ShiftRows, it produces strong diffusion.

AddRoundKey: Key Mixing

AddRoundKey XORs the current state with a round key.

$$\text{State} \leftarrow \text{State} \oplus K_i.$$

- This is the only AES step that directly uses the secret key.
- XOR is simple, fast, and reversible.
- Every round key is derived from the original cipher key.

Note

The S-box and mixing operations are public. Security must depend on the secrecy of the key, not on hiding the algorithm.

AES Key Expansion

AES does not use the same key bits unchanged in every round.

The original cipher key is expanded into several round keys:

$$K_0, K_1, K_2, \dots, K_r.$$

For AES-128:

$$r = 10, \quad \text{round keys} = 11.$$

Purpose

The key schedule ensures that each round uses key material related to, but different from, the original key.

AES-128 Round Count

AES variant	Key size	Rounds
AES-128	128 bits	10
AES-192	192 bits	12
AES-256	256 bits	14

Observation

A larger key size gives a larger key space and uses more rounds.

AES Encryption Pseudocode

```
1 # Conceptual AES encryption structure
2 state = plaintext_block
3 state = AddRoundKey(state, K0)
4
5 for i in range(1, Nr):
6     state = SubBytes(state)
7     state = ShiftRows(state)
8     state = MixColumns(state)
9     state = AddRoundKey(state, Ki)
10
11 state = SubBytes(state)
12 state = ShiftRows(state)
13 state = AddRoundKey(state, KNr)
14
15 ciphertext_block = state
```

Note

Tested cryptographic libraries exist for implementation.

Why Modes of Operation Are Needed

AES encrypts only one fixed-size block at a time:

$$\text{Enc}_K : \{0, 1\}^{128} \rightarrow \{0, 1\}^{128}.$$

But real messages are usually longer than 128 bits.

A **mode of operation** specifies how to apply a block cipher to many blocks:

$$P = P_1 \| P_2 \| \cdots \| P_m.$$

Note

A mode of operation defines how AES is repeatedly applied to multiple plaintext blocks and specifies how additional values, such as initialization vectors (IVs) and counters, are used to produce the ciphertext.

Electronic Codebook Mode

In ECB mode, each block is encrypted independently:

$$C_i = \text{Enc}_K(P_i).$$

- Same plaintext block gives same ciphertext block.
- Patterns in structured data can remain visible.
- ECB should not be used for encrypting real messages.

Problem

ECB provides confidentiality for isolated random blocks, but it leaks equality patterns across repeated blocks.

Cipher Block Chaining Mode

CBC mode chains blocks together using the previous ciphertext block.

$$C_0 = IV, \quad C_i = \text{Enc}_K(P_i \oplus C_{i-1}).$$

- Requires an unpredictable initialization vector.
- Hides repeated plaintext blocks better than ECB.
- Usually requires padding for the last block.

Decryption

$$P_i = \text{Dec}_K(C_i) \oplus C_{i-1}.$$

Counter Mode

CTR mode turns a block cipher into a keystream generator.

$$S_i = \text{Enc}_K(\text{nonce} || i), \quad C_i = P_i \oplus S_i.$$

- Encryption and decryption both use XOR.
- Blocks can be processed in parallel.
- The nonce must never repeat with the same key.

Critical rule

Reusing a nonce in CTR mode can reveal information about the plaintexts.

Padding, IVs, and Nonces

Term	Meaning
Padding	Extra bytes added when the message length is not a full block
IV	Initialization value used to randomize some modes
Nonce	Number used once; must not repeat under the same key
Counter	Value incremented to generate distinct inputs in CTR mode

Implementation warning

Many encryption failures come from incorrect IV, nonce, or padding handling, not from breaking AES itself.

Secure Communication Requirements

Secure communication usually needs more than secrecy.

- ① **Confidentiality**: attackers cannot read the message.
- ② **Integrity**: attackers cannot modify the message undetected.
- ③ **Authentication**: the receiver can verify the sender or source.
- ④ **Freshness**: old messages cannot simply be replayed as new.

Note

Encryption alone does not automatically provide message integrity.

Confidentiality vs Integrity

Confidentiality

Protects the message from being read.

$$P \xrightarrow{\text{Enc}_K} C$$

Question: *Can Eve understand it?*

Integrity

Protects the message from undetected modification.

$$(M, t) \xrightarrow{\text{verify}} \text{accept/reject}$$

Question: *Was it changed?*

Common mistake

A ciphertext can be unreadable but still malleable if no integrity protection is used.

Message Integrity

Message integrity means that unauthorized changes can be detected.

Suppose Alice sends:

$$M = \text{"Transfer 100,000 UGX"}.$$

An attacker may try to change it to:

$$M' = \text{"Transfer 900,000 UGX"}.$$

Security goal

Bob should reject modified messages unless they were produced by someone who knows the correct secret key.

Cryptographic Hash Functions

A hash function maps an input of arbitrary length to a fixed-length digest:

$$h = \text{Hash}(M).$$

- The same message always gives the same digest.
- A small message change should produce a very different digest.
- The digest is fixed length, even for long messages.

Examples

SHA-256 and SHA-3 are examples of modern cryptographic hash functions.

Security Properties of Hash Functions

A good cryptographic hash function should satisfy:

Preimage resistance

Given h , it should be hard to find M such that $\text{Hash}(M) = h$.

Second preimage resistance

Given M , it should be hard to find $M' \neq M$ with the same hash.

Collision resistance

It should be hard to find any two different messages M and M' such that $\text{Hash}(M) = \text{Hash}(M')$.

Hashing Does Not Prove Authenticity

A hash by itself is not a secret.

If an attacker changes the message, the attacker can also compute a new hash:

$$M \rightarrow M', \quad \text{Hash}(M) \rightarrow \text{Hash}(M').$$

Warning

Plain hashes detect accidental changes, but they do not authenticate the sender against an active attacker.

Solution direction

To authenticate messages, combine integrity checking with a secret key.

Message Authentication Codes

A Message Authentication Code uses a secret key to produce a tag:

$$t = \text{MAC}_K(M).$$

The receiver verifies:

$$\text{Verify}_K(M, t) \in \{\text{accept}, \text{reject}\}.$$

- Only users with the secret key can generate valid tags.
- Any message modification should cause verification to fail.
- MACs provide integrity and authentication, but not confidentiality.

HMAC: Hash-Based MAC

HMAC is a widely used method for building MACs from hash functions.

$$t = \text{HMAC}_K(M).$$

- Uses a cryptographic hash function internally.
- Uses a secret key.
- Produces a tag that the receiver can verify.

Use case

HMAC can protect API requests, protocol messages, and stored records from undetected modification.

Encrypt-then-MAC

A common secure design is to authenticate the ciphertext.

$$C = \text{Enc}_{K_1}(M), \quad t = \text{MAC}_{K_2}(C).$$

The sender transmits:

$$(C, t).$$

Receiver's order

First verify the MAC. Decrypt only if the tag is valid.

Practical rule

Do not design your own protocol unless you have expert review. Use standard authenticated encryption modes.

Authenticated Encryption

Authenticated encryption provides confidentiality and integrity together.

$$(C, t) = \text{AEAD. Enc}_K(N, A, M).$$

- N is a nonce.
- A is associated data that is authenticated but not encrypted.
- C is ciphertext.
- t is an authentication tag.

Examples

AES-GCM and ChaCha20-Poly1305 are common authenticated encryption schemes.

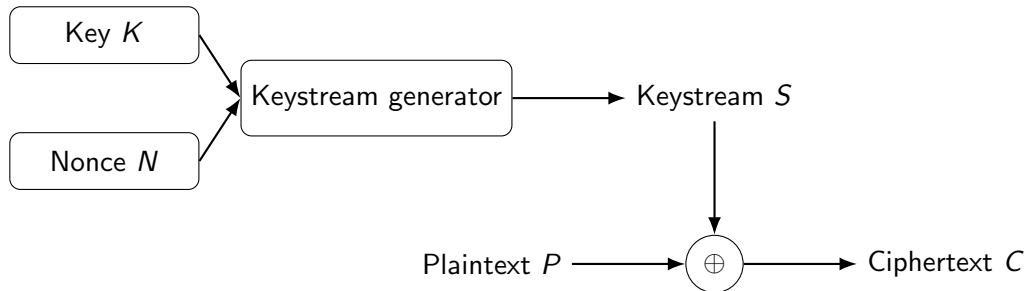
Stream Ciphers

A stream cipher encrypts data by combining plaintext with a pseudorandom keystream.

$$S = S_1 S_2 \cdots S_m, \quad C_i = P_i \oplus S_i.$$

- The keystream is generated from a secret key and nonce.
- Encryption and decryption are both XOR operations.
- Security depends on never reusing the same keystream.

Stream Cipher Model



$$C = P \oplus S, \quad P = C \oplus S.$$

Keystream Reuse Problem

If two plaintexts are encrypted with the same keystream:

$$C_1 = P_1 \oplus S, \quad C_2 = P_2 \oplus S.$$

Then an attacker can compute:

$$C_1 \oplus C_2 = (P_1 \oplus S) \oplus (P_2 \oplus S) = P_1 \oplus P_2.$$

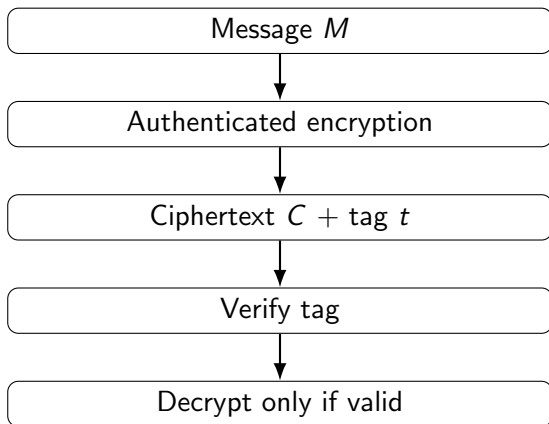
Consequence

The keystream cancels out, revealing a relationship between the two plaintexts.

Block Ciphers and Stream Ciphers Compared

Block ciphers	Stream ciphers
Encrypt fixed-size blocks	Encrypt using a keystream
Need modes for long messages	Naturally process continuous data
Examples: AES, DES	Examples: ChaCha20, historical RC4
Can be used to build stream-like modes such as CTR	Must never reuse keystream

A Modern Secure System



Modern secure systems normally combine confidentiality and integrity.

Exercise

Consider a financial transaction message:

$$M = \text{"Pay 50,000 UGX to Alice"}.$$

Discuss:

- ① What should confidentiality protect?
- ② What should integrity protect?
- ③ What could go wrong if only encryption is used?
- ④ Why is a MAC or authenticated encryption useful here?

Revision Questions

- ① Why did AES replace DES?
- ② Which AES step provides nonlinear substitution?
- ③ Why is ECB mode insecure for repeated blocks?
- ④ What is the difference between a hash and a MAC?
- ⑤ Why must stream cipher keystreams never be reused?

Questions?